Cloud and Mobile Computing

Lab 3 Report

Containerization and Cluster Orchestration

Student Name:	Bassel Adel
Group:	SE3
Lab Title:	Containerization and Cluster Orchestration

Learning Objectives

- Explain how container isolation relies on Linux namespaces and cgroups.
- Build and compare Docker images including a multi-stage build.
- Create and inspect a Kubernetes Deployment and Service in a local cluster.
- Observe scheduling decisions using node labels and node selectors.
- Test self-healing by deleting Pods and verifying automatic replacement.
- Configure readiness and liveness probes and understand their differences.

Part A: Containers, Namespaces, and cgroups

This part explores how containers use Linux namespaces for isolation and cgroups for resource control. An Ubuntu 22.04 container was launched and its isolation properties were observed by inspecting the process tree, mount points, and cgroup entries.

Task A1 — Namespace Isolation

```
docker run --rm -it --name nsdemo ubuntu:22.04 bash
hostname → a7f1bf97d721 (container-specific hostname)
ps -ef → only 2 processes: bash (PID 1) and ps (PID 10)
mount | head → overlay filesystem, cgroup2, tmpfs mounts visible
```

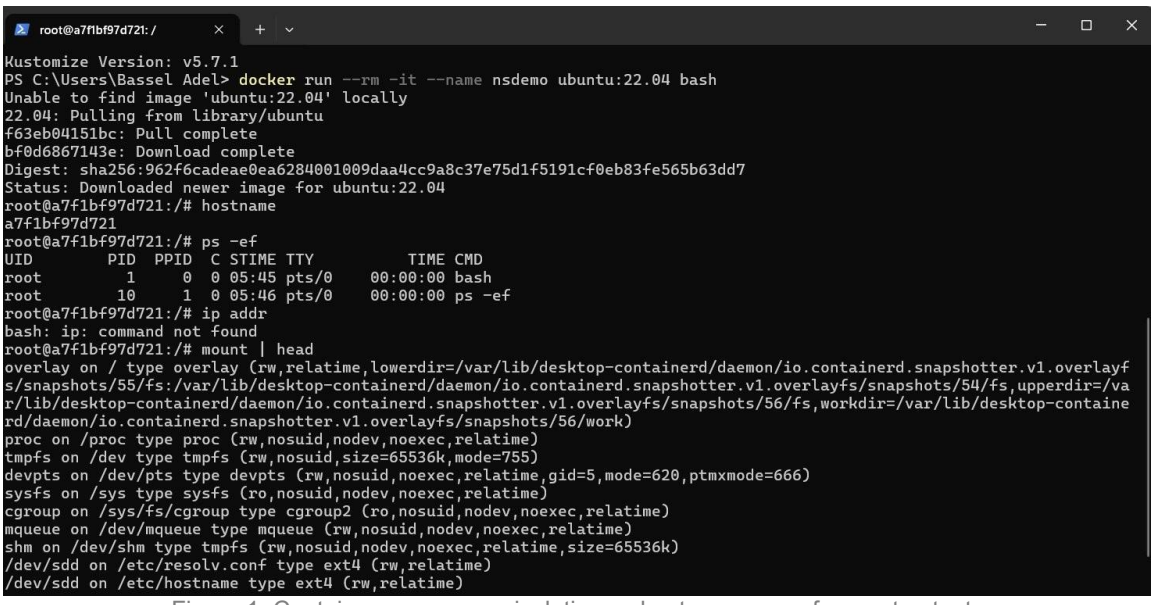

Figure 1: Container namespace isolation — hostname, ps -ef, mount output

Task A2 — Host vs Container PID Namespace

The same bash process appears as PID 1 inside the container (due to PID namespace) but has PID 1868 on the host. This demonstrates that PID namespaces create an isolated process numbering view for each container.

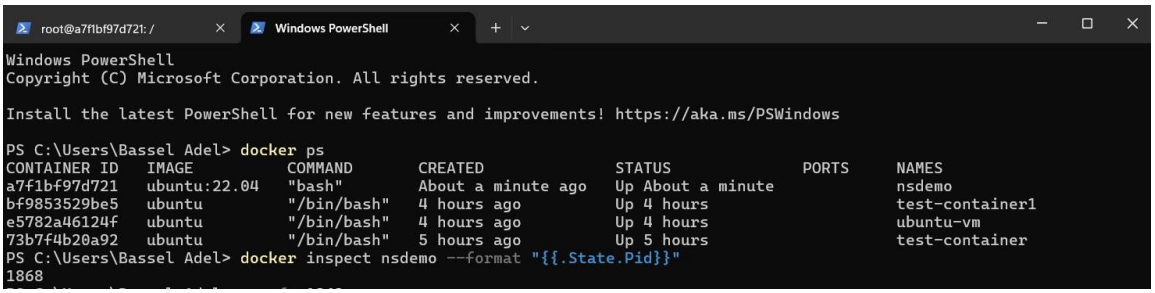

Figure 2: docker ps and inspect showing the container is running

Figure 3: Host PID 1868 mapped to container /nsdemo — PID namespace in action

Task A3 — Resource Limits with cgroups

A container was launched with --cpus=0.5 and --memory=256m. The stress tool generated CPU and memory load, and Docker Desktop Stats confirmed the cgroup limits were enforced — CPU stayed near 49% (half a core) and memory was capped at 256MB.

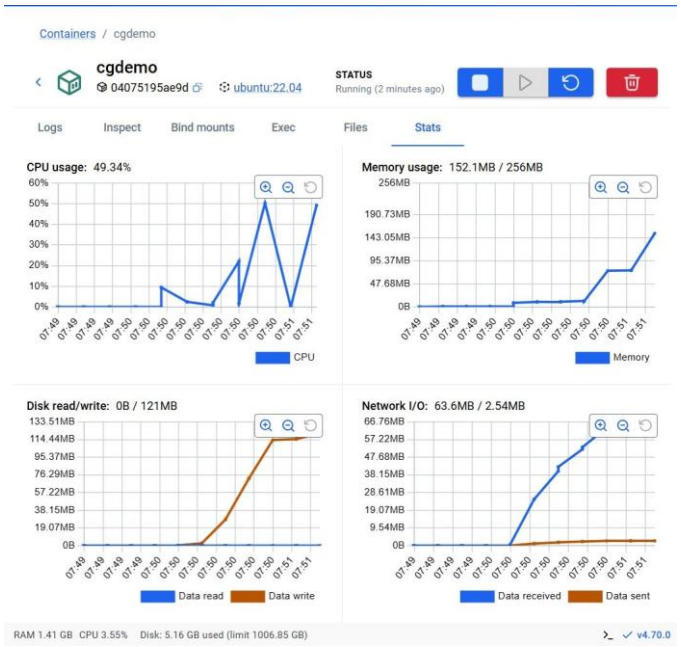

Figure 4: Docker Desktop Stats — CPU ~49% and Memory 152MB/256MB ceiling enforced by cgroups

Part B: Docker Image Layering

Two Docker images were built for a Flask application: a basic single-stage image and an optimized multi-stage image. The goal was to compare image sizes and understand how layer ordering affects caching and storage efficiency.

Image Size Comparison

```
                                                                                                  i info →   U  in use
IMAGE                                     ID             DISK USAGE   CONTENT SIZE   EXTRA
gcr.io/etcd-development/etcd:v3.5.12      cebe24b89064       84.3MB         21.6MB
lab3-basic:latest                         4dc1fc26d5a0       199MB           49MB
lab3-multi:latest                         085ac71dbb43       184MB         44.7MB
redis:alpine                              81b6f81d6a6c       134MB           35MB    U
redis:latest                              1f073813b641       204MB         55.3MB
ubuntu:22.04                              962f6cadeae0       119MB         31.7MB    U
ubuntu:latest                             c4a8d5503dfb       119MB         31.7MB    U
PS C:\Users\Bassel Adel\lab3> docker history lab3-basic
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
4dc1fc26d5a0   45 seconds ago   CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      45 seconds ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      45 seconds ago   COPY app.py . # buildkit                        12.3kB    buildkit.dockerfile.v0
<missing>      45 seconds ago   RUN /bin/sh -c pip install -r requirements.t…   16.5MB    buildkit.dockerfile.v0
<missing>      51 seconds ago   COPY requirements.txt . # buildkit              12.3kB    buildkit.dockerfile.v0
<missing>      51 seconds ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      4 hours ago      CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 hours ago      RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      4 hours ago      RUN /bin/sh -c set -eux;   savedAptMark="$(a…   41.3MB    buildkit.dockerfile.v0
<missing>      4 hours ago      ENV PYTHON_SHA256=c08bc65a81971c1dd578318282…   0B        buildkit.dockerfile.v0
<missing>      4 hours ago      ENV PYTHON_VERSION=3.12.13                      0B        buildkit.dockerfile.v0
<missing>      4 hours ago      ENV GPG_KEY=7169605F62C751356D054A26A821E680…   0B        buildkit.dockerfile.v0
<missing>      4 hours ago      RUN /bin/sh -c set -eux;  apt-get update;  a…   4.94MB    buildkit.dockerfile.v0
<missing>      4 hours ago      ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 hours ago      ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      30 hours ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.4MB    debuerreotype 0.17
```

Figure 5: docker image ls — lab3-basic (199MB) vs lab3-multi (184MB)

Layer History

```
PS C:\Users\Bassel Adel\lab3> docker history lab3-multi
IMAGE          CREATED              CREATED BY                                      SIZE      COMMENT
085ac71dbb43   35 seconds ago       CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      35 seconds ago       EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      35 seconds ago       COPY app.py . # buildkit                        12.3kB    buildkit.dockerfile.v0
<missing>      36 seconds ago       COPY /install /usr/local # buildkit             5.5MB     buildkit.dockerfile.v0
<missing>      About a minute ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      4 hours ago          CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 hours ago          RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      4 hours ago          RUN /bin/sh -c set -eux;   savedAptMark="$(a…   41.3MB    buildkit.dockerfile.v0
<missing>      4 hours ago          ENV PYTHON_SHA256=c08bc65a81971c1dd578318282…   0B        buildkit.dockerfile.v0
<missing>      4 hours ago          ENV PYTHON_VERSION=3.12.13                      0B        buildkit.dockerfile.v0
<missing>      4 hours ago          ENV GPG_KEY=7169605F62C751356D054A26A821E680…   0B        buildkit.dockerfile.v0
<missing>      4 hours ago          RUN /bin/sh -c set -eux;  apt-get update;  a…   4.94MB    buildkit.dockerfile.v0
<missing>      4 hours ago          ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 hours ago          ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      30 hours ago         # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.4MB    debuerreotype 0.17
PS C:\Users\Bassel Adel\lab3>
```

Figure 6: docker history lab3-basic — pip install layer adds 16.5MB

```
PS C:\Users\Bassel Adel\tools> kind create cluster --name lab3-cluster
Creating cluster "lab3-cluster" ...
 ✓ Ensuring node image (kindest/node:v1.29.2) 🖼
 ✓ Preparing nodes 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-lab3-cluster"
You can now use your cluster with:

kubectl cluster-info --context kind-lab3-cluster

Not sure what to do next? 😅  Check out https://kind.sigs.k8s.io/docs/user/quick-start/
PS C:\Users\Bassel Adel\tools> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:63599
CoreDNS is running at https://127.0.0.1:63599/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\Bassel Adel\tools> kubectl get nodes -o wide
NAME                 STATUS   ROLES           AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
             KERNEL-VERSION                    CONTAINER-RUNTIME
lab3-cluster-control-plane   Ready    control-plane   3m21s   v1.29.2   172.18.0.2    <none>        Debian GNU/Linux 12
(bookworm)   6.6.87.2-microsoft-standard-WSL2   containerd://1.7.13
PS C:\Users\Bassel Adel\tools>
```

Figure 7: docker history lab3-multi — only 5.5MB copied from builder stage

Property	lab3-basic	lab3-multi

Image Size	199 MB	184 MB
pip install layer	16.5 MB (kept in image)	5.5 MB (only output copied)
Build tools retained?	Yes	No — discarded after builder stage
Cache on app.py change	Re-runs pip install	Only copies new app.py layer
Security surface	Larger (build tools present)	Smaller (minimal final image)

Part C: Local Kubernetes Orchestration with kind

A local Kubernetes cluster was created using kind (Kubernetes in Docker). Both images were loaded into the cluster and a 3-replica Deployment was created with a ClusterIP Service to expose it.

Task C1 — Cluster Creation

```
PS C:\Users\Bassel Adel\lab3> kubectl apply -f deployment.yaml
deployment.apps/lab3-web created
PS C:\Users\Bassel Adel\lab3> kubectl apply -f service.yaml
service/lab3-web-svc created
PS C:\Users\Bassel Adel\lab3> kubectl get pods -o wide
NAME                        READY   STATUS    RESTARTS   AGE   IP           NODE                      NOMINATED NODE
  READINESS GATES
lab3-web-5c5f649ddc-bqghg   1/1     Running   0          10s   10.244.0.6   lab3-cluster-control-plane   <none>
  <none>
lab3-web-5c5f649ddc-n9bjz   1/1     Running   0          10s   10.244.0.5   lab3-cluster-control-plane   <none>
  <none>
lab3-web-5c5f649ddc-z8twj   1/1     Running   0          10s   10.244.0.7   lab3-cluster-control-plane   <none>
  <none>
PS C:\Users\Bassel Adel\lab3>
```

Figure 8: kind create cluster — Kubernetes v1.29.2 running on Debian GNU/Linux 12

Task C3 — Deployment

```
  <none>
PS C:\Users\Bassel Adel\lab3> kubectl get deployments
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
lab3-web   3/3     3            3           41s
PS C:\Users\Bassel Adel\lab3> kubectl describe deployment lab3-web
Name:                   lab3-web
Namespace:              default
CreationTimestamp:      Wed, 22 Apr 2026 08:10:12 +0200
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=lab3-web
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=lab3-web
  Containers:
   web:
    Image:         lab3-multi
    Port:          5000/TCP
    Host Port:     0/TCP
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
```

Figure 9: 3 pods running on lab3-cluster-control-plane with unique IPs

```
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=lab3-web
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=lab3-web
  Containers:
   web:
    Image:         lab3-multi
    Port:          5000/TCP
    Host Port:     0/TCP
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   lab3-web-5c5f649ddc (3/3 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  47s   deployment-controller  Scaled up replica set lab3-web-5c5f649ddc to 3
PS C:\Users\Bassel Adel\lab3>
```

Figure 10: kubectl describe — 3/3 replicas available, RollingUpdate strategy

ok

Figure 11: kubectl describe deployment continued — ScalingReplicaSet event confirmed

Task C4 — Service Access

Hello from Lab 3

Figure 12: Browser — 'Hello from Lab 3' at http://localhost:9090/

ok

Figure 13: Browser — 'ok' at http://localhost:9090/health

Part D: Scheduling and Placement

Node labels were applied and a nodeSelector was added to the Deployment to demonstrate declarative scheduling — Kubernetes places pods based on labels rather than explicit machine names.

Task D1 — Node Labeling

```
PS C:\Users\Bassel Adel\lab3> kubectl label nodes --all node-role=general
node/lab3-cluster-control-plane labeled
PS C:\Users\Bassel Adel\lab3> kubectl get nodes --show-labels
NAME                        STATUS   ROLES           AGE     VERSION   LABELS
lab3-cluster-control-plane   Ready    control-plane   9m53s   v1.29.2   beta.kubernetes.io/arch=amd64,beta.kubernetes.io
/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=lab3-cluster-control-plane,kubernetes.io/os=linux,node-role.ku
bernetes.io/control-plane=,node-role=general
PS C:\Users\Bassel Adel\lab3>
```

Figure 14: node-role=general label applied to lab3-cluster-control-plane

Task D2 — NodeSelector Placement

```
PS C:\Users\Bassel Adel\lab3> kubectl apply -f deployment.yaml
deployment.apps/lab3-web configured
PS C:\Users\Bassel Adel\lab3> kubectl get pods -o wide
NAME                        READY   STATUS        RESTARTS   AGE     IP            NODE                        NOMINATE
D NODE   READINESS GATES
lab3-web-585d88dd9c-7z4d2   1/1     Running       0          5s      10.244.0.9    lab3-cluster-control-plane   <none>
         <none>
lab3-web-585d88dd9c-9fskr   1/1     Running       0          4s      10.244.0.10   lab3-cluster-control-plane   <none>
         <none>
lab3-web-585d88dd9c-hskcx   1/1     Running       0          6s      10.244.0.8    lab3-cluster-control-plane   <none>
         <none>
lab3-web-5c5f649ddc-bqghg   1/1     Terminating   0          4m34s   10.244.0.6    lab3-cluster-control-plane   <none>
         <none>
lab3-web-5c5f649ddc-n9bjz   1/1     Terminating   0          4m34s   10.244.0.5    lab3-cluster-control-plane   <none>
         <none>
lab3-web-5c5f649ddc-z8twj   1/1     Terminating   0          4m34s   10.244.0.7    lab3-cluster-control-plane   <none>
         <none>
PS C:\Users\Bassel Adel\lab3>
```

Figure 15: After nodeSelector added — old pods terminating, new pods scheduled on labeled node

Reflection: Using nodeSelector with labels like node-role=general is fundamentally different from hard-coding a machine name. Labels are abstract — any node matching the label can host the pods. If a node is replaced, adding the same label to the new node is enough for pods to schedule there. Hard-coded hostnames would require manual YAML edits for every node change.

Part E: Self-Healing and Health Probes

This part demonstrates Kubernetes reconciliation — the platform continuously compares desired state with actual state and corrects divergence automatically.

Task E1 — Self-Healing by Pod Deletion

```
PS C:\Users\Bassel Adel\lab3> kubectl get pods
NAME                          READY   STATUS        RESTARTS   AGE
lab3-web-585d88dd9c-7z4d2     1/1     Running       0          32s
lab3-web-585d88dd9c-9fskr     1/1     Running       0          31s
lab3-web-585d88dd9c-hskcx     1/1     Running       0          33s
lab3-web-5c5f649ddc-bqghg     0/1     Terminating   0          5m1s
PS C:\Users\Bassel Adel\lab3> kubectl delete pod lab3-web-5c5f649ddc-bqghg
Error from server (NotFound): pods "lab3-web-5c5f649ddc-bqghg" not found
PS C:\Users\Bassel Adel\lab3>
```

Figure 16: Pod deleted — 3 brand new replacement pods created immediately

```
PS C:\Users\Bassel Adel\lab3> kubectl get pods
NAME                          READY   STATUS        RESTARTS   AGE
lab3-web-585d88dd9c-7z4d2     1/1     Running       0          32s
lab3-web-585d88dd9c-9fskr     1/1     Running       0          31s
lab3-web-585d88dd9c-hskcx     1/1     Running       0          33s
lab3-web-5c5f649ddc-bqghg     0/1     Terminating   0          5m1s
PS C:\Users\Bassel Adel\lab3> kubectl delete pod lab3-web-5c5f649ddc-bqghg
Error from server (NotFound): pods "lab3-web-5c5f649ddc-bqghg" not found
PS C:\Users\Bassel Adel\lab3> kubectl get pods -w
NAME                          READY   STATUS    RESTARTS   AGE
lab3-web-585d88dd9c-7z4d2     1/1     Running   0          79s
lab3-web-585d88dd9c-9fskr     1/1     Running   0          78s
lab3-web-585d88dd9c-hskcx     1/1     Running   0          80s
```

Figure 17: kubectl get pods -w — old pod Terminating while new pod reaches Running

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka
.ms/PSWindows

PS C:\Users\Bassel Adel> $env:PATH += ";C:\Users\Bassel Adel\tools"
PS C:\Users\Bassel Adel> kubectl get pods -w
NAME                          READY   STATUS    RESTARTS   AGE
lab3-web-585d88dd9c-7z4d2     1/1     Running   0          4m37s
lab3-web-585d88dd9c-9fskr     1/1     Running   0          4m36s
lab3-web-585d88dd9c-hskcx     1/1     Running   0          4m38s
lab3-web-probe-f5dc5585b-kg9vt   1/1     Running   0          114s
lab3-web-probe-f5dc5585b-znknp   1/1     Running   0          114s
lab3-web-probe-f5dc5585b-kg9vt   1/1     Terminating   0          3m42s
lab3-web-probe-f5dc5585b-qgq8b   0/1     Pending       0          0s
lab3-web-probe-f5dc5585b-qgq8b   0/1     Pending       0          0s
lab3-web-probe-f5dc5585b-qgq8b   0/1     ContainerCreating   0          0s
lab3-web-probe-f5dc5585b-qgq8b   0/1     Running             0          1s
lab3-web-probe-f5dc5585b-qgq8b   1/1     Running             0          6s
lab3-web-probe-f5dc5585b-kg9vt   0/1     Terminating         0          4m12
s
lab3-web-probe-f5dc5585b-kg9vt   0/1     Terminating         0          4m13
s
lab3-web-probe-f5dc5585b-kg9vt   0/1     Terminating         0          4m13
s
lab3-web-probe-f5dc5585b-kg9vt   0/1     Terminating         0          4m13
s
```

Figure 18: Real-time watch — Pending → ContainerCreating → 0/1 Running → 1/1 Running in seconds

Task E2 — Readiness and Liveness Probes

```
Windows PowerShell
PS C:\Users\Bassel Adel\lab3> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lab3-web-585d88dd9c-7z4d2           1/1     Running   0          3m23s
lab3-web-585d88dd9c-9fskr           1/1     Running   0          3m22s
lab3-web-585d88dd9c-hskcx           1/1     Running   0          3m24s
lab3-web-probe-f5dc5585b-kg9vt      1/1     Running   0          40s
lab3-web-probe-f5dc5585b-znknp      1/1     Running   0          40s
PS C:\Users\Bassel Adel\lab3> kubectl describe pod lab3-web-probe-f5dc5585b-kg9vt
Name:             lab3-web-probe-f5dc5585b-kg9vt
Namespace:        default
Priority:         0
Service Account:  default
Node:             lab3-cluster-control-plane/172.18.0.2
Start Time:       Wed, 22 Apr 2026 08:17:24 +0200
Labels:           app=lab3-web-probe
                  pod-template-hash=f5dc5585b
Annotations:      <none>
Status:           Running
IP:               10.244.0.12
IPs:
  IP:           10.244.0.12
Controlled By:  ReplicaSet/lab3-web-probe-f5dc5585b
Containers:
  web:
    Container ID:   containerd://d98c3de99f11ed220d99f77c88b60ec513c367c2eb7f6b33d1589aaa2452306c
    Image:          lab3-multi
    Image ID:       docker.io/library/import-2026-04-22@sha256:085ac71dbb432b8a7344b9fc10ac3256c470177cd4965cf6f4c87cd6e
6a8b1f7
    Port:           5000/TCP
    Host Port:      0/TCP
```

Figure 19: kubectl describe pod — all conditions True, probes configured

```
Windows PowerShell
    State:          Running
      Started:      Wed, 22 Apr 2026 08:17:25 +0200
    Ready:          True
    Restart Count:  0
    Liveness:       http-get http://:5000/health delay=10s timeout=1s period=10s #success=1 #failure=3
    Readiness:      http-get http://:5000/health delay=3s timeout=1s period=5s #success=1 #failure=3
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-4bjgv (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-4bjgv:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
```

Figure 20: Liveness: delay=10s period=10s | Readiness: delay=3s period=5s on /health endpoint

Task E3 — Container Process Kill

```
PS C:\Users\Bassel Adel\lab3> kubectl exec -it lab3-web-probe-f5dc5585b-kg9v
t -- /bin/sh
# kill 1
# kill 1
# kill -9 1
# kill -SIGKILL 1
/bin/sh: 4: kill: Illegal option -S
# exit
command terminated with exit code 2
PS C:\Users\Bassel Adel\lab3> kubectl delete pod lab3-web-probe-f5dc5585b-kg
9vt
pod "lab3-web-probe-f5dc5585b-kg9vt" deleted from default namespace
PS C:\Users\Bassel Adel\lab3>
```

Figure 21: Pod deleted after kill attempts — Kubernetes replaced it automatically

```
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  45s   default-scheduler  Successfully assigned default/lab3-web-probe-f5dc5585b-kg9vt to lab3-clust
er-control-plane
  Normal  Pulled     45s   kubelet            Container image "lab3-multi" already present on machine
  Normal  Created    45s   kubelet            Created container web
  Normal  Started    44s   kubelet            Started container web
PS C:\Users\Bassel Adel\lab3>
```

Figure 22: Events — Scheduled → Pulled → Created → Started lifecycle confirmed

Reflection Questions

Q: Why do namespaces alone not guarantee fair resource use?

A: Namespaces only control visibility — they isolate what a process can see (filesystem, network, PIDs). They do not limit CPU or memory consumption. A process in its own namespace could still use 100% of host CPU. cgroups are needed to enforce resource limits.

Q: How do cgroups improve cluster stability?

A: cgroups prevent any single container from monopolizing CPU or memory. By setting limits (as shown in Task A3 with --cpus=0.5 and --memory=256m), the scheduler can make reliable placement decisions and the node remains stable even under heavy workloads.

Q: Why is Docker image layering important for large-scale orchestration?

A: Layer caching dramatically reduces build times and network transfer. If only app.py changes, the pip install layer is reused. In a cluster with hundreds of nodes, shared base layers mean only changed layers need downloading — saving time and bandwidth at scale.

Q: What does Kubernetes mean by desired state?

A: Desired state is the configuration declared in YAML (e.g., replicas: 3). Kubernetes continuously reconciles actual state with desired state. If a pod dies, the controller detects the gap and creates a replacement — exactly as observed in Task E1.

Q: How is self-healing different from traditional manual operations?

A: Traditional operations require a human to detect failures and manually restart services. Kubernetes self-healing is automatic and continuous — the Deployment controller watches pod state at all times and reconciles without human intervention, dramatically reducing MTTR.

Q: Why are readiness and liveness probes not interchangeable?

A: Readiness probes control traffic routing — a failing pod is removed from Service endpoints but keeps running. Liveness probes control container restart — a failing pod is killed and restarted. Mixing them up causes either unnecessary restarts or broken containers still receiving no traffic but never being fixed.

Q: What is one limitation of a single-node kind cluster vs a real multi-node cluster?

A: In a single-node cluster all pods land on the same node regardless of nodeSelector rules — there is no real scheduling decision. In a multi-node cluster the scheduler must balance load, respect constraints, and handle node failures, where declarative scheduling truly shines.

Conclusion

This lab provided hands-on experience with the full containerization and orchestration stack. Starting from Linux primitives (namespaces and cgroups), through Docker image layering, to Kubernetes deployment, scheduling, and self-healing — each layer builds on the previous. The key insight is that

Kubernetes abstracts away individual machines and lets developers declare intent, while the platform continuously enforces it. This declarative, self-healing model is what makes cloud-native systems resilient and manageable at scale.